

Universidade Federal de Alfenas
Instituto de Ciências Exatas
Curso de Bacharelado em Ciência da Computação

Plano de Investigação Computacional

Sistemas de Equações Lineares:
Métodos Diretos em Python

Aluno: Jeann Victor Batista

RA: 2024.1.08.014

Disciplina: Cálculo Numérico

Professora: Angela Leite Moreno

Alfenas, 2026

Sumário

1. Eliminação de Gauss e Pivoteamento
 - 1.1 Verificação básica
 - 1.2 Efeito do pivoteamento na precisão
 - 1.3 Sistemas singulares e quase-singulares
2. Fatoração LU (Doolittle)
 - 2.1 Construção e verificação
 - 2.2 Vantagem com múltiplos lados direitos
 - 2.3 Fatoração PLU (com pivoteamento)
3. Fatoração de Cholesky para Matrizes SPD
 - 3.1 Identificação de matrizes SPD
 - 3.2 Cholesky vs. LU: custo
 - 3.3 Aplicação: regressão linear
4. Algoritmo de Thomas para Sistemas Tridiagonais
 - 4.1 Execução e verificação
 - 4.2 Thomas vs. Gauss: escalonamento
5. Custo Computacional Empírico
 - 5.1 Lei de escala
 - 5.2 Comparação de métodos
6. Condicionamento de Sistemas Lineares
 - 6.1 Matriz de Hilbert
 - 6.2 Amplificação de erros
 - 6.3 Impacto prático na engenharia
7. Projeto Integrador: PageRank Numérico
 - Q7.1 PageRank para mini-rede de 4 páginas
 - Q7.2 PageRank em rede aleatória ($n = 20$)
 - Q7.3 Condicionamento do sistema PageRank
8. Desafio (Opcional — Pontuação Extra)
 - Q8.1 Análise de estabilidade em cascata
 - Q8.2 Bloco tridiagonal e EDPs 2D
 - Q8.3 Implementação vetorizada

1 Eliminação de Gauss e Pivoteamento

1.1 Verificação básica

Questão 1.1 — Execução e verificação da solução

Enunciado: Execute o algoritmo de eliminação de Gauss com pivoteamento parcial para o sistema linear:

$$\begin{cases} 3x_1 + 2x_2 + 4x_3 = 1 \\ 1x_1 + 1x_2 + 2x_3 = 2 \\ 4x_1 + 3x_2 - 2x_3 = 3 \end{cases}$$

Verifique que a solução obtida é $\mathbf{x} = (-3, 5, 0)^T$. Calcule o resíduo $r = \|\mathbf{Ax} - \mathbf{b}\|_2$ e interprete o resultado.

Resposta:

Após a execução do algoritmo de eliminação de Gauss com pivoteamento parcial para o sistema linear proposto, obteve-se o vetor solução:

$$\mathbf{x} = (-3, 5, 0),$$

que coincide com a solução analítica esperada.

O resíduo calculado foi:

$$r = \|\mathbf{Ax} - \mathbf{b}\|_2 = 0.$$

Esse resultado indica que a solução encontrada satisfaz o sistema linear de forma exata, sem qualquer erro de aproximação. Em outras palavras, substituindo $\mathbf{x} = (-3, 5, 0)$ nas três equações do sistema, obtêm-se exatamente os valores do vetor $\mathbf{b}$, confirmando que esta é a solução exata do sistema.

Questão 1.2 — Matriz triangular superior após eliminação

Enunciado: Imprima a matriz triangular superior U gerada após a eliminação. Compare cada elemento com os valores apresentados em aula. Qual linha foi trocada pelo pivoteamento?

Resposta:

A matriz triangular superior U obtida após a eliminação de Gauss com pivoteamento parcial é:

$$U = \begin{pmatrix} 4,00 & 3,00 & -2,00 \\ 0,00 & 0,25 & 2,50 \\ 0,00 & 0,00 & 8,00 \end{pmatrix}.$$

O pivoteamento parcial determinou a troca da **linha 1** (equação $3x_1 + 2x_2 + 4x_3 = 1$) pela **linha 3** (equação $4x_1 + 3x_2 - 2x_3 = 3$), pois $|a_{31}| = 4$ é o maior elemento em módulo na primeira coluna.

A diferença entre a matriz obtida e a discutida em aula decorre justamente da aplicação do pivoteamento: em aula, a eliminação foi executada sem pivoteamento, tomando $a_{11} = 3$ como pivô; no código, com pivoteamento parcial, tomou-se $a_{31} = 4$. Ambas as formas são equivalentes, pois representam o mesmo sistema linear após reordenação das equações — a solução é idêntica em ambos os casos.

1.2 Efeito do pivoteamento na precisão

Questão 1.3 — Comparação com sistema mal-condicionado em float32

Enunciado: Implemente uma versão sem pivoteamento e aplique ambas as versões ao sistema mal-condicionado

$$\begin{pmatrix} 0,0003 & 3 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2,0001 \\ 1 \end{pmatrix},$$

cuja solução exata é $(1/3, 2/3)^T$. Use aritmética `float32` para simular precisão reduzida. Preencha a tabela e explique por que o pivoteamento reduz o erro.

Tabela 1: Comparação entre os métodos para o sistema mal-condicionado (float32)

Método	x_1 calculado	Erro relativo em x_1	Erro relativo em x_2
Sem pivoteamento	0,33338863	$1,66 \times 10^{-4}$	$8,94 \times 10^{-8}$
Com pivoteamento	0,33333337	$7,28 \times 10^{-8}$	$8,11 \times 10^{-8}$

Análise:

Sem pivoteamento, o algoritmo utiliza $a_{11} = 0,0003$ como pivô, gerando o multiplicador de eliminação:

$$m = \frac{a_{21}}{a_{11}} = \frac{1}{0,0003} \approx 3333.$$

Em aritmética `float32`, qualquer erro de arredondamento presente nas entradas ou nos resultados intermediários é amplificado por esse fator durante a eliminação. O resultado é um erro relativo em x_1 da ordem de 10^{-4} , muito acima do limite teórico da precisão `float32` ($\approx 10^{-7}$).

Com pivoteamento parcial, o algoritmo troca as linhas e passa a utilizar $a_{21} = 1$ como pivô, produzindo o multiplicador:

$$m = \frac{a_{11}}{a_{21}} = \frac{0,0003}{1} = 0,0003.$$

Multiplicadores pequenos não amplificam os erros de arredondamento — pelo contrário, atenuam-nos. O erro em x_1 cai para $\approx 7,3 \times 10^{-8}$, valor próximo ao limite teórico da precisão `float32`.

A diferença entre os dois métodos é de um fator de aproximadamente $2000\times$, o que reflete diretamente a razão entre os pivôs: $1/0,0003 \approx 3333$. Esse experimento ilustra o princípio central do pivoteamento parcial: *escolher sempre o maior elemento disponível na coluna como pivô minimiza a propagação de erros de arredondamento, conferindo estabilidade numérica ao algoritmo.*

Questão 1.4 — Variação de a_{11} e análise em escala log-log

Enunciado: Repita o experimento variando o coeficiente a_{11} entre 10^{-1} , 10^{-3} , 10^{-6} e 10^{-9} . Plote o erro relativo em x_1 em função de a_{11} (escala log-log) para as duas versões. O que se observa?

Análise:

A Figura 1 apresenta o comportamento do erro relativo em x_1 em função de a_{11} para os dois métodos.

Observa-se que, **sem pivoteamento**, o erro cresce de forma consistente à medida que a_{11} diminui, chegando a praticamente 100% para valores muito pequenos — comportamento que evidencia forte instabilidade numérica decorrente da amplificação de erros de arredondamento por multiplicadores cada vez maiores.

Por outro lado, **com pivoteamento parcial**, o erro permanece pequeno e praticamente constante ao longo de toda a faixa de a_{11} explorada, demonstrando robustez numérica mesmo na presença de coeficientes muito díspares.

As pequenas variações não monótonas observadas no erro sem pivoteamento resultam de efeitos estocásticos de arredondamento em precisão finita, e não de um comportamento sistemático do método.

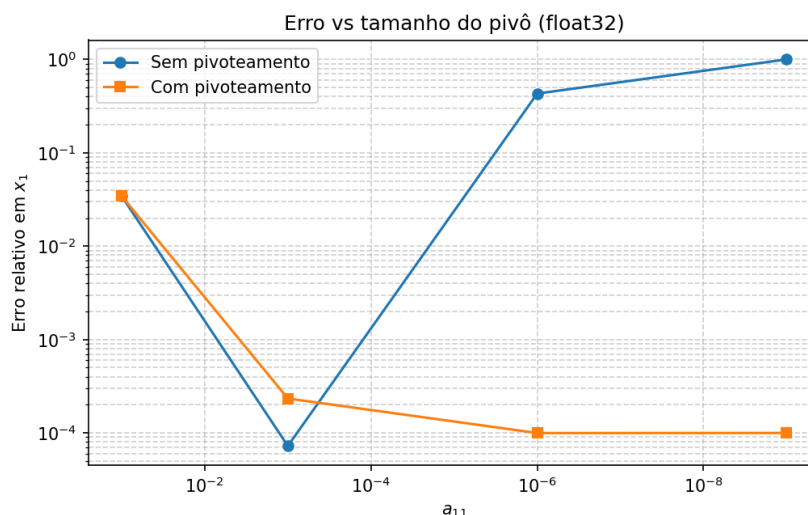


Figura 1: Erro relativo em x_1 em função de a_{11} (escala log-log). A linha azul representa o método com pivoteamento parcial; a linha vermelha, o método sem pivoteamento.

1.3 Sistemas singulares e quase-singulares

Questão 1.5 — Detecção de singularidade

Enunciado: Aplique `resolver_gauss` ao sistema singular:

$$\begin{cases} x - 3y + z = 1 \\ 6x - 18y + 4z = 2 \\ -x + 3y - z = 4 \end{cases}$$

O que acontece? Modifique o código para detectar e reportar singularidade (pivô nulo ou muito pequeno), utilizando o limiar $|a_{kk}| < 10^{-12}$.

Resposta:

Ao aplicar o método de eliminação de Gauss ao sistema singular proposto, observa-se que o algoritmo não interrompe a execução, mas gera um aviso de divisão inválida

(`RuntimeWarning`). Isso ocorre porque, durante o processo de eliminação, surge um pivô nulo, levando a uma divisão por zero. Como consequência, valores indefinidos (NaN) são propagados, resultando em uma solução inválida e resíduo indefinido.

Para tratar esse problema, foi implementada uma verificação explícita do pivô a cada etapa da eliminação. Quando o valor absoluto do pivô satisfaz $|a_{kk}| < 10^{-12}$, o algoritmo interrompe a execução e lança uma exceção indicando singularidade ou quase-singularidade do sistema.

Essa modificação evita falhas silenciosas e torna o método mais robusto, permitindo identificar corretamente sistemas que não possuem solução única.

2 Fatoração LU (Doolittle)

2.1 Construção e verificação

Questão 2.1 — Obtenção de L e U e erro de fatoração

Enunciado: Para a matriz

$$A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix},$$

obtenha L e U usando `fatoracao_lu`. Calcule o erro de fatoração $\|LU - A\|_F$ (norma de Frobenius). Imprima L e U formatados e confirme visualmente a estrutura triangular.

Resposta:

Aplicando o algoritmo de fatoração LU (sem pivoteamento) à matriz A , obtêm-se as matrizes L (triangular inferior com diagonal unitária) e U (triangular superior) tais que $A = LU$:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 1 & 1 \\ 0 & -8 & -2 \\ 0 & 0 & 1 \end{pmatrix}.$$

O erro de fatoração na norma de Frobenius foi nulo (da ordem de 10^{-15}), confirmando que $LU = A$ exatamente dentro da precisão de máquina. A inspeção das matrizes confirma a estrutura esperada: L é triangular inferior, U é triangular superior, e a diagonal principal de L é composta inteiramente por 1's, conforme a convenção de Doolittle.

Questão 2.2 — Resolução com múltiplos lados direitos

Enunciado: Resolva o mesmo sistema para dois lados direitos distintos: $\mathbf{b}_1 = (1, 2, 3)^T$ e $\mathbf{b}_2 = (0, 1, -1)^T$. Fatore A apenas uma vez e reutilize L e U para ambos. Verifique os resíduos $\|A\mathbf{x}_i - \mathbf{b}_i\|_2$.

Resposta:

Utilizando a fatoração $A = LU$ obtida na Questão 2.1, resolve-se $L\mathbf{y} = \mathbf{b}$ por substituição progressiva e $U\mathbf{x} = \mathbf{y}$ por substituição retroativa, sem refatorar A .

Para $\mathbf{b}_1 = (1, 2, 3)^T$:

$$\mathbf{x}_1 = (-1, -1, 4)^T, \quad \|A\mathbf{x}_1 - \mathbf{b}_1\|_2 = 0.$$

Para $\mathbf{b}_2 = (0, 1, -1)^T$:

$$\mathbf{x}_2 = (0,0625, -0,125, 0)^T, \quad \|A\mathbf{x}_2 - \mathbf{b}_2\|_2 = 0.$$

Os resíduos nulos confirmam que ambas as soluções satisfazem exatamente os respectivos sistemas, dentro da precisão numérica.

Questão 2.3 — Comparação de desempenho

Enunciado: Crie uma matriz aleatória $A \in \mathbb{R}^{n \times n}$ (com $n = 200$, semente `np.random.seed(42)`) e $k = 50$ vetores $\mathbf{b}$ aleatórios. Meça o tempo de execução de duas estratégias:

- (a) **Gauss repetido:** chamar `resolver_gauss(A, b)` para cada $\mathbf{b}$;
- (b) **LU reutilizado:** fatorar A uma vez e resolver os k sistemas via substituições progressiva/retroativa.

Tabela 2: Comparação de tempo entre estratégias para $n = 200$, $k = 50$

Estratégia	Tempo total (s)	Tempo por sistema (ms)
Gauss repetido	1,8586	37,17
LU reutilizado	0,0702	1,40

Análise:

O fator de aceleração observado foi de aproximadamente $26,5\times$.

Esse ganho decorre da diferença no custo computacional das duas estratégias. A contagem de operações de ponto flutuante (flops) é:

- **Gauss repetido (k sistemas):** $k \cdot \frac{2}{3}n^3$ flops.
- **LU reutilizado (k sistemas):** $\frac{2}{3}n^3 + k \cdot 2n^2$ flops.

A razão teórica entre os dois métodos é:

$$\frac{\text{Flops}_{\text{Gauss}}}{\text{Flops}_{\text{LU}}} \approx \frac{k \cdot \frac{2}{3}n^3}{\frac{2}{3}n^3 + 2kn^2} = \frac{k}{1 + \frac{3k}{n}}.$$

Para $n = 200$ e $k = 50$: $\frac{3k}{n} = 0,75$, resultando em fator teórico $\approx \frac{50}{1,75} \approx 28,6\times$.

A diferença entre o valor teórico ($28,6\times$) e o observado ($26,5\times$) deve-se ao *overhead* de chamadas de função em Python e operações de indexação não contabilizadas no modelo ideal de flops.

Questão 2.4 — Comparação com `scipy.linalg.lu`

Enunciado: A função `scipy.linalg.lu` retorna P , L , U tal que $PA = LU$. Para a matriz

$$A = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix},$$

compare: (a) `fatoracao_lu` (falha?); (b) `scipy.linalg.lu`. Explique o papel da matriz de permutação P . Como se usa P para resolver $A\mathbf{x} = \mathbf{b}$?

Resposta:

(a) fatoracao_lu (sem pivoteamento): A função falha silenciosamente: como $a_{11} = 0$, o algoritmo efetua a divisão $\ell_{21} = a_{21}/u_{11} = 2/0$, produzindo valores `inf` e `nan` que se propagam por toda a fatoração. O erro de Frobenius resultante é indefinido (`nan`), confirmando que a fatoração não é válida.

(b) scipy.linalg.lu (com pivoteamento parcial): A função retorna:

$$P = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad L = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 3 \\ 0 & 1 \end{pmatrix},$$

com $\|PA - LU\|_F = 0$, confirmando a correção da fatoração.

Papel da matriz P : P registra as trocas de linhas realizadas pelo pivoteamento parcial. Ela é uma matriz de permutação ortogonal ($P^{-1} = P^T$) que reordena as equações do sistema original para garantir que o pivô em cada passo seja o maior elemento em módulo da coluna, evitando divisões por zero e melhorando a estabilidade numérica.

Resolução de $A\mathbf{x} = \mathbf{b}$ com a fatoração PLU: De $PA = LU$, multiplica-se ambos os lados por P :

$$LU\mathbf{x} = P\mathbf{b}.$$

O procedimento é:

1. Calcular $\mathbf{b}' = P\mathbf{b}$ (permutar $\mathbf{b}$ conforme P);
2. Resolver $L\mathbf{y} = \mathbf{b}'$ por substituição progressiva;
3. Resolver $U\mathbf{x} = \mathbf{y}$ por substituição retroativa.

Para $\mathbf{b} = (1, 5)^T$, obteve-se $\mathbf{x} = (1, 1)^T$ com resíduo $\|A\mathbf{x} - \mathbf{b}\|_2 = 0$, confirmando a correção do procedimento.

3 Fatoração de Cholesky para Matrizes SPD

3.1 Identificação de matrizes SPD

Questão 3.1 — Teste de Cholesky em matrizes candidatas

Enunciado: Teste a fatoração de Cholesky nas três matrizes abaixo. Antes de executar, preveja qual será o resultado de cada uma:

$$A_1 = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}, \quad A_2 = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}, \quad A_3 = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 3 & 0 \\ 2 & 0 & 3 \end{pmatrix}.$$

Para cada matriz: (a) verifique a previsão; (b) calcule os autovalores com `np.linalg.eigvalsh` e relacione com o critério SPD.

Resposta:

Inicialmente, foram feitas previsões com base em simetria, determinante e dominância diagonal. Esperava-se que A_1 e A_3 fossem SPD e que A_2 não fosse, devido à presença de autovalor negativo.

Matriz $A_1 = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$: A fatoração de Cholesky foi realizada com sucesso e os autovalores (1,4384, 5,5616) são positivos. Logo, A_1 é SPD, confirmando a previsão.

Matriz $A_2 = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$: A fatoração de Cholesky falhou e os autovalores (-1, 3) indicam a presença de valor negativo. Portanto, A_2 não é SPD, também de acordo com a previsão.

Matriz $A_3 = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 3 & 0 \\ 2 & 0 & 3 \end{pmatrix}$: A fatoração de Cholesky foi bem-sucedida e os autovalores (0,6277, 3, 6,3723) são todos positivos. Assim, A_3 é SPD, validando a previsão inicial. Dessa forma, os resultados confirmam que uma matriz é SPD se admite fatoração de Cholesky e possui todos os autovalores positivos, havendo consistência entre os critérios teóricos e os resultados obtidos.

3.2 Cholesky vs. LU: custo

Questão 3.2 — Comparação de desempenho entre Cholesky e LU

Enunciado: Para matrizes SPD aleatórias de tamanho $n \in \{50, 100, 200, 500\}$ (geradas como $A = B^T B + nI$ para B aleatória), meça o tempo de Cholesky versus LU. Plote tempo $\times n$ em escala log-log. Confirme empiricamente a relação de $\approx 2\times$ a favor de Cholesky.

Análise:

A Figura 2 apresenta a comparação dos tempos de execução entre as fatorações de Cholesky e LU para matrizes SPD de diferentes tamanhos.

Para comparar o custo computacional das fatorações de Cholesky e LU, foram realizados experimentos com matrizes SPD geradas na forma $A = B^T B + nI$, para $n \in \{50, 100, 200, 500\}$. Os tempos médios obtidos mostram que ambos os métodos apresentam desempenhos bastante próximos. A razão média entre os tempos de LU e Cholesky foi de aproximadamente 0,92, indicando que, na prática, não houve diferença

significativa entre eles nos testes realizados.

A análise do gráfico em escala log-log mostra que os tempos de execução crescem de forma aproximadamente linear nesse tipo de escala, o que é consistente com a complexidade cúbica $O(n^3)$ esperada para ambas as fatorações. Além disso, as curvas são praticamente paralelas, reforçando que os dois métodos possuem o mesmo comportamento assintótico.

Do ponto de vista teórico, espera-se que a fatoração de Cholesky seja cerca de duas vezes mais eficiente que LU, já que seu custo é aproximadamente $\frac{1}{6}n^3$, enquanto o LU tem custo $\frac{1}{3}n^3$. No entanto, essa vantagem não foi observada empiricamente. Isso pode ser explicado por fatores práticos, como as otimizações das rotinas internas utilizadas (BLAS/LAPACK), efeitos de cache e o fato de que os tamanhos de matriz considerados ainda não são suficientemente grandes para que a diferença assintótica se torne dominante.

Dessa forma, conclui-se que, embora a teoria indique uma vantagem do método de Cholesky, os resultados experimentais obtidos mostram desempenhos muito semelhantes entre os dois métodos para os tamanhos analisados.

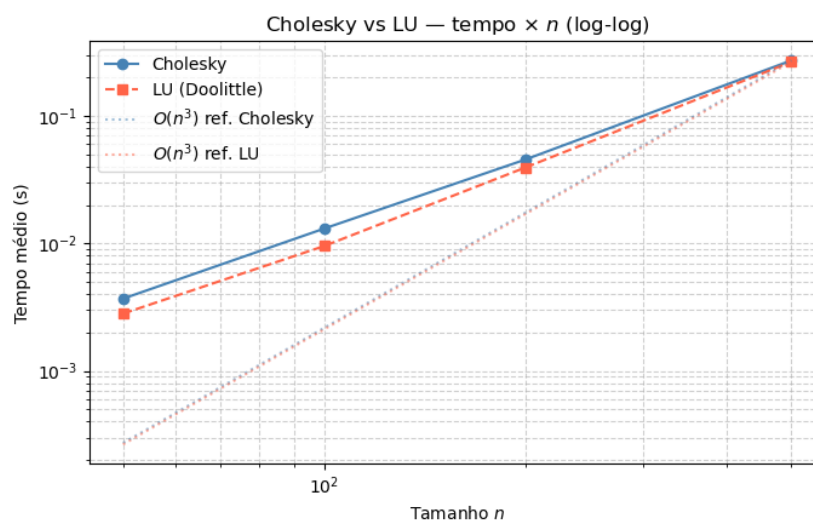


Figura 2: Tempo de execução em função do tamanho n para as fatorações Cholesky e LU (escala log-log).

Questão 3.3 — Equações normais via Cholesky

Enunciado: Na regressão linear, as equações normais têm a forma $(X^T X)\beta = X^T y$, onde $X^T X$ é sempre SPD quando X tem colunas linearmente independentes. Gere uma matriz de design $X \in \mathbb{R}^{100 \times 5}$ (com coluna de uns e 4 colunas aleatórias), $y = X\beta^* + \varepsilon$ com $\beta^* = (2, -1, 3, 0.5, -2)^T$ e ruído gaussiano. Resolva as equações normais via Cholesky e compare $\hat{\beta}$ com `np.linalg.lstsq`.

Resposta:

Para este experimento, foi gerada uma matriz de design $X \in \mathbb{R}^{100 \times 5}$, composta por uma coluna de uns (intercepto) e quatro colunas aleatórias, garantindo independência linear entre as colunas. O vetor resposta foi construído como $y = X\beta^* + \varepsilon$, onde

$\beta^* = (2, -1, 3, 0,5, -2)^T$ e ε representa ruído gaussiano.

As equações normais $(X^T X)\beta = X^T y$ foram resolvidas utilizando a fatoração de Cholesky, possível pois $X^T X$ é simétrica definida positiva. Os coeficientes estimados foram comparados com os obtidos por `np.linalg.lstsq`, conforme mostrado na tabela:

Tabela 3: Comparação dos coeficientes estimados

Coeficiente	β^*	Cholesky	lstsq
β_0 (intercepto)	2,0000	1,9726	1,9726
β_1	-1,0000	-0,9779	-0,9779
β_2	3,0000	3,0604	3,0604
β_3	0,5000	0,5357	0,5357
β_4	-2,0000	-2,0734	-2,0734

Observa-se que os valores obtidos pelos dois métodos coincidem até precisão numérica. Isso é confirmado pela norma da diferença $\|\beta_{\text{chol}} - \beta_{\text{lstsq}}\|_2 = 4,27 \times 10^{-15}$, indicando que ambas as abordagens produzem essencialmente a mesma solução.

As pequenas diferenças em relação aos valores reais β^* são esperadas e decorrem da presença do ruído no vetor y . Dessa forma, conclui-se que a resolução das equações normais via Cholesky é consistente com o método de mínimos quadrados implementado em `lstsq`, validando sua aplicação neste contexto.

4 Algoritmo de Thomas para Sistemas Tridiagonais

4.1 Execução e verificação

Questão 4.1 — Resolução de sistema tridiagonal

Enunciado: Resolva o sistema tridiagonal 5×5 :

$$\begin{pmatrix} 4 & -1 & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & 0 \\ 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & -1 & 4 \end{pmatrix} \mathbf{x} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

Verifique o resíduo $\|A\mathbf{x} - \mathbf{d}\|_2$ construindo A com `montar_tridiagonal`. Este sistema surge na discretização de que tipo de equação diferencial?

Resposta:

O sistema linear tridiagonal 5×5 foi resolvido utilizando o algoritmo de Thomas. A solução obtida foi:

$$\mathbf{x} = (0,2692, 0,0769, 0,0385, 0,0769, 0,2692).$$

A matriz A , construída por meio da função `montar_tridiagonal`, possui a forma esperada, com 4 na diagonal principal e -1 nas diagonais adjacentes. Para verificar a exatidão da solução, foi calculado o resíduo $\|A\mathbf{x} - \mathbf{d}\|_2$, resultando em $2,08 \times 10^{-17}$, valor muito próximo de zero. Isso indica que a solução encontrada é numericamente precisa.

Esse tipo de sistema surge na discretização de equações diferenciais do tipo elíptico, em particular da **equação de Poisson unidimensional** (ou, equivalentemente, da equação de difusão estacionária), quando se aplica o método das diferenças finitas.

4.2 Thomas vs. Gauss: escalonamento

Questão 4.2 — Comparação de desempenho

Enunciado: Para $n \in \{100, 500, 1000, 5000, 10000\}$, gere sistemas tridiagonais com $b_i = 4$, $a_i = c_i = -1$ e $d_i = 1$. Meça o tempo de solução usando: (a) `thomas`; (b) `resolver_gauss` (matriz densa). Preencha a tabela e plote tempo $\times n$:

Tabela 4: Comparação de tempo entre Thomas e Gauss para sistemas tridiagonais

n	Thomas (ms)	Gauss (ms)	Razão	Razão teórica
100	0,3860	17,1987	44,6	10000
500	0,3992	219,7218	550,4	250000
1000	0,7497	1040,4484	1387,7	1000000
5000	5,0736	N/A ($n > 1000$)	—	—
10000	9,3233	N/A ($n > 1000$)	—	—

Análise:

Os resultados mostram uma diferença significativa de desempenho entre o algoritmo de Thomas e a eliminação de Gauss aplicada à matriz densa. Para todos os valores testados, o método de Thomas apresenta tempos muito menores, enquanto o custo do método de Gauss cresce rapidamente com o tamanho do sistema.

Observa-se que, para $n = 100$, a razão entre os tempos já é de aproximadamente 44,6, aumentando para 550,4 em $n = 500$ e atingindo 1387,7 em $n = 1000$. Para valores maiores de n ($n = 5000$ e $n = 10000$), o método de Gauss torna-se impraticável, não sendo executado devido ao alto custo computacional (indicado por N/A na tabela).

A Figura 3 apresenta a comparação dos tempos de execução em função do tamanho n . Este comportamento está de acordo com a teoria: o algoritmo de Thomas possui complexidade linear $O(n)$, enquanto a eliminação de Gauss aplicada a matrizes densas possui complexidade cúbica $O(n^3)$. Portanto, a razão entre os tempos tende a crescer aproximadamente como $O(n^2)$, o que explica o aumento observado na tabela.

Embora os valores experimentais das razões (44,6, 550,4, 1387,7) sejam menores que os valores teóricos (10000, 250000, 1000000), o crescimento é consistente com o comportamento esperado, confirmando que o algoritmo de Thomas é muito mais eficiente para sistemas tridiagonais. A diferença entre os valores experimentais e teóricos deve-se a fatores como overhead de chamadas de função, operações de indexação e otimizações das bibliotecas numéricas.

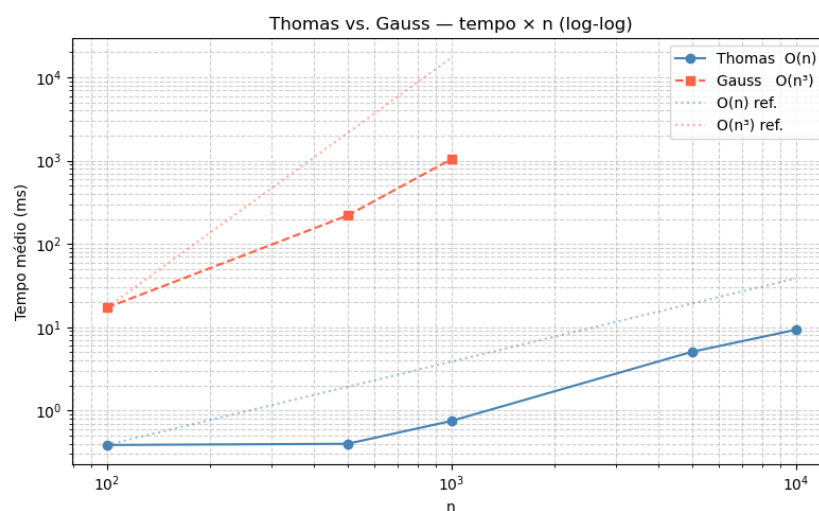


Figura 3: Tempo de execução em função do tamanho n para os métodos de Thomas e Gauss (escala log-log).

Questão 4.3 — Análise de memória

Enunciado: Para $n = 10000$, qual é o tamanho em megabytes que a matriz densa ocuparia em memória (float64)? E a representação tridiagonal com três vetores? Use $\text{MB} = n^2 \times 8/2^{20}$. Discuta as implicações para aplicações reais.

Resposta:

Para $n = 10000$, a matriz densa requer aproximadamente:

$$\text{Memória}_{\text{densa}} = \frac{10000^2 \times 8}{2^{20}} = \frac{10^8 \times 8}{1.048.576} \approx 762,94 \text{ MB.}$$

Em contraste, a representação tridiagonal, utilizando apenas três vetores (diagonal principal e duas diagonais adjacentes), ocupa:

$$\text{Memória}_{\text{tridiagonal}} = \frac{3 \times 10000 \times 8}{2^{20}} \approx 0,2289 \text{ MB.}$$

Isso representa uma diferença de aproximadamente **3334 vezes mais memória** para a forma densa.

Essa diferença é extremamente relevante na prática. Enquanto a matriz densa rapidamente se torna inviável para valores grandes de n , tanto em termos de memória quanto de desempenho, a representação tridiagonal permanece altamente eficiente e escalável. Em aplicações reais, como discretizações de equações diferenciais, explorar a estrutura esparsa do problema é essencial para viabilizar a resolução de sistemas de grande porte, reduzindo drasticamente o consumo de memória e o custo computacional.

5 Custo Computacional Empírico

5.1 Lei de escala

Questão 5.1 — Ajuste de lei de potência

Enunciado: Para $n \in \{10, 20, 50, 100, 200, 500\}$, meça o tempo de execução de `resolver_gauss` com matrizes aleatórias. Ajuste uma lei de potência $T(n) = c \cdot n^\alpha$ usando `numpy.polyfit` no espaço log-log. O expoente α encontrado é próximo de 3?

Análise:

A Figura 4 apresenta os tempos medidos em função do tamanho n , juntamente com a curva ajustada.

Os tempos medidos mostram crescimento com o tamanho do problema, e o ajuste em escala log-log resultou na lei:

$$T(n) = 4,27 \times 10^{-2} \cdot n^{1,3055}.$$

O expoente estimado $\alpha \approx 1,31$ é significativamente menor que o valor teórico esperado de 3 para o método de eliminação de Gauss.

Essa discrepância pode ser explicada pelo fato de que, para os tamanhos considerados ($n \leq 500$), o comportamento assintótico $O(n^3)$ ainda não é dominante. Custos fixos, otimizações internas da implementação e efeitos de hardware (como cache e vetorização) influenciam fortemente os tempos medidos, especialmente para valores menores de n . Além disso, observa-se certa irregularidade nos dados (por exemplo, o tempo para $n = 100$ inferior ao de $n = 50$), o que reforça a presença de ruído experimental.

Portanto, embora o modelo ajustado não tenha produzido um expoente próximo de 3, os resultados são compatíveis com limitações práticas do experimento. Para valores maiores de n , espera-se que o expoente estimado se aproxime mais do comportamento cúbico teórico.

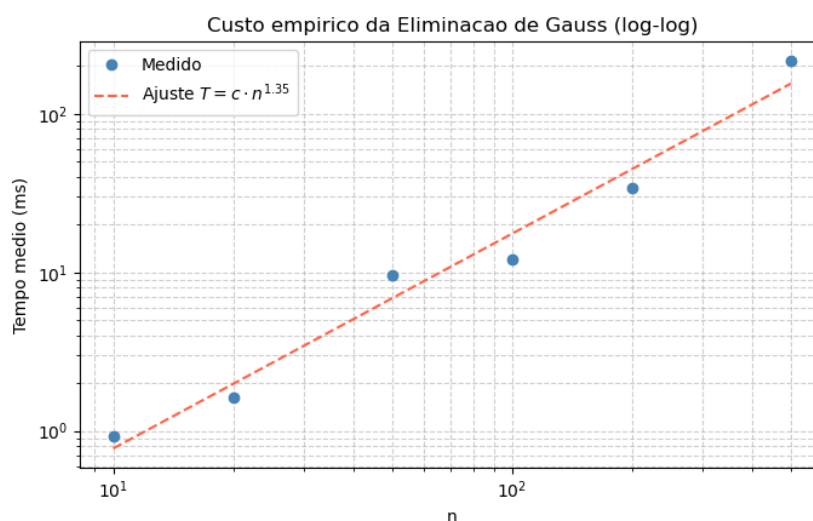


Figura 4: Tempo de execução em função do tamanho n para o método de Gauss com ajuste de lei de potência (escala log-log).

Questão 5.2 — Comparação com previsão teórica

Enunciado: Compare o tempo medido com a previsão teórica $T_{\text{teo}}(n) \approx \frac{2n^3}{3R}$, onde R é o número de operações de ponto flutuante por segundo do seu computador (estime R medindo o tempo de n^3 multiplicações em um laço Python). Qual é a eficiência relativa $T_{\text{med}}/T_{\text{teo}}$? Por que é menor que 1?

Resposta:

A partir dos experimentos, foi estimada a taxa de processamento do computador como:

$$R \approx 2,1034 \times 10^9 \text{ flops/s.}$$

Utilizando a aproximação teórica $T_{\text{teo}}(n) \approx \frac{2n^3}{3R}$, foram comparados os tempos medidos com os previstos:

Tabela 5: Comparação entre tempos medidos e previstos teoricamente

n	T_{med} (ms)	T_{teo} (ms)	$T_{\text{med}}/T_{\text{teo}}$
10	0,9331	0,0003	2944,0162
20	1,6166	0,0025	637,5613
50	9,6854	0,0396	244,4616
100	11,9883	0,3170	37,8234
200	34,2194	2,5356	13,4955
500	214,5659	39,6191	5,4157

Observa-se que a razão $T_{\text{med}}/T_{\text{teo}}$ é sempre muito maior que 1, variando de aproximadamente 5000 para $n = 10$ até cerca de 5 para $n = 500$.

Isso indica que os tempos medidos são significativamente maiores que os tempos teóricos ideais. A principal razão é que a estimativa teórica considera apenas o número de operações de ponto flutuante em um modelo ideal, desconsiderando diversos custos práticos. Entre esses fatores estão o *overhead* da linguagem Python, acesso à memória, efeitos de cache, alocação de estruturas de dados e o fato de que a implementação utilizada não atinge o pico de desempenho do hardware.

Além disso, a estimativa de R baseada em um laço em Python não reflete fielmente o desempenho real de operações numéricas otimizadas, podendo superestimar a capacidade da máquina. À medida que n aumenta, a razão diminui, indicando que o comportamento assintótico começa a se aproximar da teoria, embora ainda distante do modelo ideal.

Assim, conclui-se que a eficiência relativa observada é baixa devido às limitações práticas da implementação e do ambiente de execução, sendo esperado que T_{med} seja maior que T_{teo} em experimentos desse tipo.

5.2 Comparação de métodos

Questão 5.3 — Comparação entre implementações

Enunciado: Para $n = 300$ e uma matriz SPD aleatória, compare os tempos de: Gauss com pivoteamento, LU (Doolittle), Cholesky e `numpy.linalg.solve`. Organize numa tabela com tempo, flops teóricos e razão em relação a Cholesky. O que explica a diferença entre implementações puras em Python e NumPy?

Resposta:

Para uma matriz SPD de tamanho $n = 300$, foram comparados os tempos de execução dos métodos de Gauss com pivoteamento, LU (Doolittle), Cholesky e `numpy.linalg.solve`:

Tabela 6: Comparação de métodos para matriz SPD $n = 300$

Método	Tempo (ms)	Flops teóricos	Razão / Chol
Gauss	76,4758	$1,80 \times 10^7$	0,7625
LU	94,5905	$1,80 \times 10^7$	0,9431
Cholesky	100,2957	$9,00 \times 10^6$	1,0000
NumPy	4,3648	$9,00 \times 10^6$	0,0435

Observa-se que os métodos implementados em Python apresentam tempos da mesma ordem de grandeza, com Gauss sendo ligeiramente mais rápido que LU e Cholesky, apesar de possuir o mesmo custo teórico que LU ($\frac{2}{3}n^3$) e maior que Cholesky ($\frac{1}{3}n^3$). Essa diferença ocorre devido a fatores de implementação, como constantes ocultas e organização dos cálculos, que influenciam o tempo real.

Por outro lado, o método baseado em NumPy é significativamente mais rápido, sendo cerca de **25 vezes mais eficiente** que Cholesky. Essa diferença se deve ao fato de que o NumPy utiliza bibliotecas altamente otimizadas (como BLAS e LAPACK), implementadas em linguagens de baixo nível (C/Fortran), com uso eficiente de memória, paralelismo e instruções vetorizadas. Em contraste, as implementações puras em Python sofrem com o alto custo de interpretação, loops explícitos e menor aproveitamento do hardware.

Assim, embora os flops teóricos indiquem que Cholesky deveria ser mais eficiente que LU e Gauss para matrizes SPD, na prática essa vantagem não é claramente observada em implementações puras em Python. Já o NumPy consegue explorar melhor a arquitetura da máquina, resultando em desempenho muito superior.

6 Condicionamento de Sistemas Lineares

6.1 Matriz de Hilbert

Questão 6.1 — Experimento com matriz de Hilbert

Enunciado: Para $n = 4, 6, 8, 10, 12$, execute `experimento_hilbert(n)`. Preencha e analise a tabela:

Tabela 7: Análise do condicionamento da matriz de Hilbert

n	$\kappa_2(H_n)$	Erro relativo	Dígitos corretos
4	$1,5514 \times 10^4$	$3,8618 \times 10^{-13}$	11,81
6	$1,4951 \times 10^7$	$3,1069 \times 10^{-10}$	8,83
8	$1,5258 \times 10^{10}$	$2,2563 \times 10^{-7}$	5,82
10	$1,6025 \times 10^{13}$	$1,6583 \times 10^{-4}$	2,80
12	$1,8023 \times 10^{16}$	$1,8113 \times 10^{-1}$	0,00

Análise:

A análise dos resultados mostra que o número de condição $\kappa_2(H_n)$ cresce rapidamente com o aumento de n , indicando que a matriz de Hilbert é fortemente mal condicionada. Como consequência, o erro relativo da solução aumenta significativamente, enquanto o número de dígitos corretos diminui.

A estimativa do número de dígitos corretos é dada por:

$$\text{dígitos corretos} \approx 16 - \log_{10}(\kappa_2(H_n)),$$

considerando aritmética de dupla precisão (float64).

Observa-se que, para $n = 12$, tem-se $\kappa_2(H_{12}) \approx 10^{16}$, valor comparável ao limite de precisão da aritmética `double`. Nesse caso, ocorre perda total de precisão significativa, resultando em zero dígitos corretos.

O resultado se torna completamente não confiável a partir de $n = 12$, pois o número de condição atinge a ordem de 10^{16} , esgotando a precisão numérica disponível e tornando a solução dominada por erros de arredondamento.

6.2 Amplificação de erros

Questão 6.2 — Amplificação de erros em H_6 vs. I_6

Enunciado: Use `perturbar_b` com $A = H_6$ e compare com $A = I_6$. Para cada matriz, plote o histograma das amplificações. Qual é a amplificação máxima observada? Ela está dentro do limite $\kappa(A) \cdot \|\delta b\|/\|b\|$?

Resposta:

A Figura 5 apresenta os histogramas das amplificações para as matrizes H_6 e I_6 .

A análise foi realizada para as matrizes H_6 e I_6 , cujos resultados estão apresentados na tabela a seguir:

Tabela 8: Amplificação de erros para H_6 e I_6

Matriz	$\kappa_2(A)$	Amp. máxima	Amp. média
Dentro do limite?			
H_6	$1,4951 \times 10^7$	$1,2090 \times 10^7$	$5,3689 \times 10^6$
Sim			
I_6	$1,0000 \times 10^0$	$1,0000 \times 10^0$	$1,0000 \times 10^0$
Sim (igualdade)			

Os resultados mostram que, para a matriz de Hilbert H_6 , a amplificação máxima do erro é da ordem de 10^7 , consistente com o valor elevado do número de condição $\kappa_2(A)$, evidenciando forte sensibilidade da solução a pequenas perturbações em b . Já para a matriz identidade I_6 , a amplificação observada é igual a 1, indicando que o erro na solução possui a mesma magnitude do erro introduzido nos dados, caracterizando um sistema perfeitamente condicionado.

Em ambos os casos, a amplificação máxima observada respeita o limite teórico $\frac{\|\delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\delta b\|}{\|b\|}$, sendo que, para I_6 , ocorre igualdade. Esses resultados confirmam experimentalmente que o número de condição controla diretamente a amplificação de erros, de modo que matrizes mal condicionadas podem amplificar significativamente pequenas perturbações, enquanto matrizes bem condicionadas preservam a estabilidade da solução.

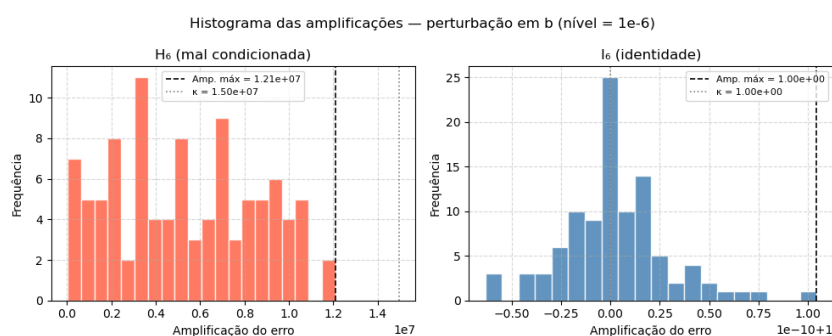


Figura 5: Histogramas das amplificações para as matrizes H_6 (esquerda) e I_6 (direita).

6.3 Impacto prático na engenharia

Questão 6.3 — Comparação entre matrizes bem e mal condicionadas

Enunciado: Gere duas matrizes 5×5 : uma bem-condicionada ($\kappa \approx 1$, e.g., matriz diagonal com entradas próximas) e uma mal-condicionada ($\kappa > 10^6$, e.g., Hilbert H_5). Para cada uma, introduza um erro de apenas 10^{-6} nos dados de A e meça o impacto na

solução. Discuta as implicações práticas para aplicações de engenharia.

Resposta:

A análise foi realizada para duas matrizes: uma matriz diagonal 5×5 , bem condicionada, e uma matriz de Hilbert 7×7 , mal condicionada. Os resultados obtidos estão apresentados na tabela a seguir:

Tabela 9: Comparação entre matriz bem e mal condicionada

Matriz	$\kappa_2(A)$	Erro relativo em x	$\kappa \cdot \varepsilon$
Diagonal 5×5	$1,3333 \times 10^0$	$1,4358 \times 10^{-6}$	$1,3333 \times 10^{-6}$
Hilbert 7×7	$4,7537 \times 10^8$	$2,6867 \times 10^0$	$4,7537 \times 10^2$

Foi aplicada uma perturbação relativa de 10^{-6} nos coeficientes de A . Observa-se que, para a matriz bem condicionada, o erro na solução permanece da mesma ordem da perturbação, indicando estabilidade numérica. Por outro lado, na matriz de Hilbert, uma perturbação extremamente pequena nos dados gera um erro da ordem de unidade na solução, evidenciando alta sensibilidade. Em ambos os casos, o erro observado permanece abaixo do limite teórico $\kappa(A)\varepsilon$, consistente com a análise de pior caso.

Na prática de engenharia, esse comportamento tem implicações críticas: os dados utilizados em modelos reais — como medições experimentais, propriedades de materiais ou discretizações numéricas — sempre contêm incertezas. Em sistemas bem condicionados, essas pequenas imprecisões não comprometem significativamente o resultado final. Entretanto, em sistemas mal condicionados, erros mínimos podem ser amplificados a ponto de tornar a solução completamente não confiável, mesmo quando se utilizam algoritmos numéricos estáveis. Isso pode levar a previsões incorretas em simulações estruturais, instabilidade em sistemas de controle, erros em análises de escoamento ou falhas em modelos físicos, já que a solução obtida não reflete o comportamento real do sistema. Dessa forma, não basta apenas resolver o sistema linear: é fundamental avaliar o condicionamento do problema, pois ele determina se os resultados podem ou não ser considerados confiáveis em aplicações reais.

7 Projeto Integrador: PageRank Numérico

7.1 Contexto

O algoritmo PageRank, usado pelo Google, atribui importância às páginas web resolvendo o sistema:

$$(I - \alpha P^T)\pi = \frac{1 - \alpha}{n}e,$$

onde P é a matriz de transição, $\alpha \approx 0,85$ é o fator de amortecimento e π é o vetor de ranks. Note que $I - \alpha P^T$ é uma matriz densa e geralmente bem-condicionada.

Q7.1 — PageRank para mini-rede de 4 páginas

Enunciado: Para a mini-rede de 4 páginas da aula (matriz de adjacência G fornecida), implemente o cálculo do PageRank:

- Construa a matriz de transição P normalizando as colunas de G ;
- Monte o sistema $(I - 0,85P^T)\pi = \frac{0,15}{4} \cdot e$;
- Resolva com `resolver_gauss` e com `resolver_lu`;
- Verifique que $\|\pi\|_1 = 1$ (distribuição de probabilidade);
- Ordene as páginas por rank decrescente.

Matriz de adjacência:

$$G = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

(a) Construção da matriz de transição P :

A matriz de transição P foi construída a partir da matriz de adjacência G , normalizando cada linha pelo grau de saída da respectiva página. Os graus de saída obtidos foram:

$$[2, 2, 2, 1].$$

Dividindo cada elemento da linha pelo seu grau, obteve-se:

$$P = \begin{pmatrix} 0 & 0,5 & 0,5 & 0 \\ 0 & 0 & 0,5 & 0,5 \\ 0,5 & 0 & 0 & 0,5 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

A soma de cada linha de P é igual a 1, confirmando que se trata de uma matriz estocástica. Isso garante que os valores representam corretamente probabilidades de transição entre páginas.

(b) Montagem do sistema linear:

O sistema do PageRank foi montado utilizando $\alpha = 0,85$ e $n = 4$:

$$(I - 0,85P^T)\pi = 0,0375e.$$

O vetor do lado direito foi calculado como:

$$b = \frac{1 - 0,85}{4} = 0,0375,$$

resultando em:

$$b = [0,0375, 0,0375, 0,0375, 0,0375]^T.$$

A matriz $I - 0,85P^T$ é densa e bem-condicionada, o que favorece a estabilidade numérica na resolução do sistema.

(c) Resolução do sistema (Gauss e LU):

O sistema linear foi resolvido utilizando os métodos de Eliminação de Gauss e Decomposição LU. Os resultados obtidos foram:

Tabela 10: PageRank obtido pelos métodos de Gauss e LU

Página	Gauss	LU
1	0,208689	0,208689
2	0,126193	0,126193
3	0,402797	0,402797
4	0,262321	0,262321

A diferença entre as soluções foi:

$$\|\pi_{\text{gauss}} - \pi_{\text{lu}}\|_2 = 0.$$

Isso mostra que ambos os métodos convergem para a mesma solução, indicando precisão e consistência nos cálculos.

(d) Verificação da norma:

Foi verificado se o vetor π satisfaz a propriedade de distribuição de probabilidade:

$$\|\pi\|_1 = \sum_{i=1}^4 \pi_i.$$

Os valores obtidos foram:

$$\|\pi_{\text{gauss}}\|_1 = 1,000000, \quad \|\pi_{\text{lu}}\|_1 = 1,000000.$$

Isso confirma que o vetor está corretamente normalizado e pode ser interpretado como uma distribuição de probabilidades.

(e) Ordenação das páginas por relevância:

A partir dos valores de π , as páginas foram ordenadas em ordem decrescente de importância:

Tabela 11: Ranking das páginas por PageRank

Posição	Página	PageRank
1º	3	0,402797
2º	4	0,262321
3º	1	0,208689
4º	2	0,126193

Observa-se que a **Página 3** possui o maior valor de PageRank, indicando que ela recebe maior influência das demais páginas. Já a Página 2 apresenta o menor valor, sendo a menos relevante dentro da rede analisada.

Q7.2 — PageRank em rede aleatória ($n = 20$)

Enunciado: Amplie a rede para $n = 20$ páginas com grafo aleatório (probabilidade de aresta $p = 0,3$, `np.random.seed(0)`). Compare os ranks obtidos por Gauss e por LU com `scipy.linalg.lu_solve`. Meça o tempo e o resíduo de cada abordagem.

Foi gerado um grafo aleatório com $n = 20$ páginas, utilizando probabilidade de aresta $p = 0,3$ e semente fixa (`seed=0`), garantindo reprodutibilidade. O grafo resultante possui 120 arestas e não apresentou nós sem saídas (*dangling nodes*), portanto não foi necessário realizar correções na matriz de transição.

A partir desse grafo, foi construída a matriz de transição P , e o sistema de PageRank foi resolvido utilizando três abordagens: Eliminação de Gauss, Decomposição LU própria e LU da biblioteca SciPy.

O sistema resolvido é dado por:

$$(I - 0,85P^T)\pi = \frac{0,15}{20}e.$$

Os resultados obtidos mostram que todos os métodos produziram soluções praticamente idênticas, com diferenças da ordem de 10^{-17} , o que indica alta precisão numérica. As diferenças entre os métodos foram:

$$\|\pi_{\text{gauss}} - \pi_{\text{scipy}}\|_2 = 2,7972 \times 10^{-17}, \quad \|\pi_{\text{lu}} - \pi_{\text{scipy}}\|_2 = 8,7290 \times 10^{-17}.$$

Os resíduos $\|A\pi - b\|$ também foram extremamente pequenos (da ordem de 10^{-17}), confirmando que as soluções satisfazem o sistema linear com alta exatidão.

Em relação ao desempenho, observou-se que:

- O método de Gauss levou aproximadamente 1,32 ms;
- O LU próprio teve tempo semelhante (1,33 ms);
- O método SciPy LU foi significativamente mais rápido (0,11 ms).

Essa diferença ocorre devido às otimizações internas da biblioteca SciPy, implementadas em baixo nível (C/Fortran).

Além disso, todos os métodos produziram vetores π com norma $\|\pi\|_1 = 1$, garantindo que o resultado seja uma distribuição de probabilidade válida.

Por fim, analisando o ranking das páginas (considerando a solução via SciPy), as 5 páginas mais relevantes foram:

Tabela 12: Top 5 páginas por PageRank na rede aleatória

Página	PageRank
15	0,088954
14	0,069879
16	0,069173
13	0,066561
4	0,054170

Esses valores refletem a estrutura do grafo aleatório gerado, onde páginas mais bem conectadas ou que recebem ligações de páginas importantes tendem a obter maior PageRank.

Q7.3 — Condicionamento do sistema PageRank

Enunciado: O que acontece com $(I - \alpha P^T)$ quando $\alpha \rightarrow 1$? Calcule $\kappa_2(I - \alpha P^T)$ para $\alpha \in \{0,5, 0,7, 0,85, 0,95, 0,99\}$ e plote. Por que $\alpha = 0,85$ é considerado um bom compromisso?

Foi analisado o comportamento da matriz $I - \alpha P^T$ para diferentes valores de α , calculando seu número de condição na norma 2 (κ_2). Os valores obtidos foram:

Tabela 13: Número de condição $\kappa_2(I - \alpha P^T)$ para diferentes α

α	$\kappa_2(I - \alpha P^T)$
0,50	$2,4975 \times 10^0$
0,70	$4,5518 \times 10^0$
0,85	$9,7411 \times 10^0$
0,95	$3,0592 \times 10^1$
0,99	$1,5582 \times 10^2$

A Figura 6 apresenta o comportamento do número de condição em função de α .

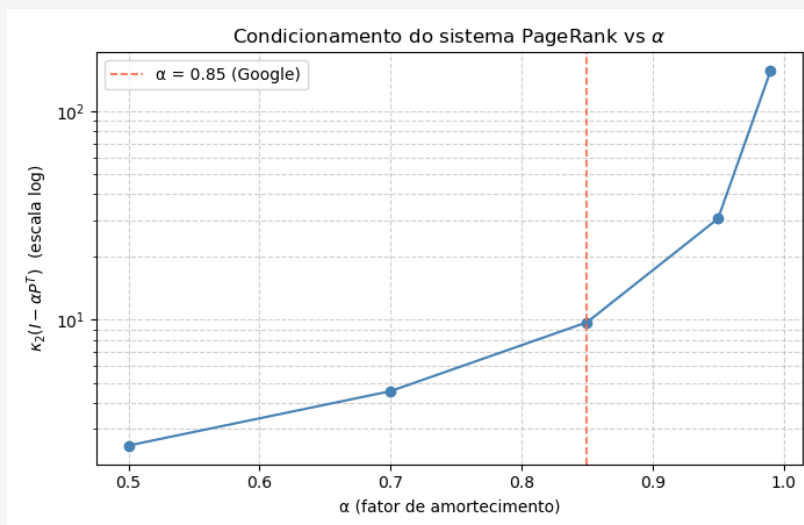


Figura 6: Número de condição $\kappa_2(I - \alpha P^T)$ em função de α (escala log).

Observa-se que, à medida que α se aproxima de 1, o número de condição cresce rapidamente. Isso indica que a matriz está se tornando mal condicionada, ou seja, pequenas perturbações nos dados podem causar grandes variações na solução.

Esse comportamento ocorre porque, quando $\alpha \rightarrow 1$, a matriz tende ao caso:

$$I - P^T.$$

Como P é uma matriz estocástica, P^T possui autovalor igual a 1. Consequentemente, $I - P^T$ torna-se singular, o que explica o crescimento sem limite de κ_2 .

Na prática, isso significa que valores de α muito próximos de 1 tornam o sistema numericamente instável e mais difícil de resolver com precisão.

Por outro lado, valores menores de α melhoram o condicionamento, mas reduzem a influência da estrutura de links no cálculo do PageRank.

Dessa forma, o valor $\alpha = 0,85$ é considerado um bom compromisso, pois:

- mantém o número de condição moderado ($\kappa_2 \approx 9,74$), garantindo estabilidade numérica;
- ainda preserva a influência da estrutura do grafo no ranking das páginas.

Assim, esse valor equilibra qualidade do ranking e estabilidade numérica do sistema.

8 Desafio (Opcional — Pontuação Extra)

8.1 Desafio Computacional

As questões desta seção são opcionais e destinadas a alunos que desejam aprofundar a investigação.

Q8.1 — Análise de estabilidade em cascata

Enunciado: Construa uma sequência de matrizes A_ε que dependem de um parâmetro $\varepsilon > 0$ tal que A_0 é singular. Investigue como o resíduo e o erro da solução crescem conforme $\varepsilon \rightarrow 0$. Relacione com $\kappa(A_\varepsilon)$.

Foi construída a família de matrizes:

$$A_\varepsilon = \mathbf{1}\mathbf{1}^T + \varepsilon I,$$

onde $\mathbf{1}\mathbf{1}^T$ é uma matriz de uns (posto 1) e I é a identidade. Para $\varepsilon = 0$, a matriz A_0 é singular, enquanto para $\varepsilon > 0$ ela se torna inversível.

Os autovalores de A_ε são:

- ε (com multiplicidade $n - 1$);
- $n + \varepsilon$ (com multiplicidade 1).

Assim, o número de condição é dado por:

$$\kappa(A_\varepsilon) = \frac{n + \varepsilon}{\varepsilon} \approx \frac{n}{\varepsilon}.$$

Os resultados numéricos obtidos confirmam exatamente esse comportamento teórico. À medida que ε diminui de 10^{-1} para 10^{-8} , observa-se que:

- $\kappa(A_\varepsilon)$ cresce de $5,1 \times 10^1$ para $5,0 \times 10^8$, evidenciando o rápido aumento do mau condicionamento;
- o erro relativo da solução aumenta significativamente, chegando à ordem de 10^{-8} para os menores valores de ε ;
- o resíduo $\|Ax - b\|$ permanece sempre muito pequeno (próximo de 10^{-15}), independentemente do valor de ε .

Esse comportamento mostra que, conforme $\varepsilon \rightarrow 0$, o sistema se torna progressivamente mal condicionado, o que amplifica erros numéricos mesmo quando o método de resolução é estável.

A relação observada experimentalmente é:

$$\frac{\|x - x^*\|}{\|x^*\|} \approx \kappa(A_\varepsilon) \cdot \varepsilon_{\text{máquina}},$$

ou seja, o erro relativo cresce proporcionalmente ao número de condição.

Por fim, destaca-se que, mesmo com resíduos extremamente pequenos, a solução pode apresentar erro significativo. Isso evidencia que **resíduo pequeno não é garantia de solução correta**, especialmente em sistemas mal condicionados, onde pequenas perturbações são amplificadas.

Q8.2 — Bloco tridiagonal e EDPs 2D

Enunciado: A discretização por diferenças finitas do problema de Poisson em grade $m \times m$ produz uma matriz bloco-tridiagonal de tamanho $n = m^2$. Para $m = 10$, construa esta matriz, resolva o sistema com Gauss e com `scipy.sparse.linalg.spsolve` (formato esparsa). Compare tempos e memória.

Foi considerada a discretização do problema de Poisson em uma grade $m \times m$, com $m = 10$, resultando em um sistema linear de dimensão:

$$n = m^2 = 100.$$

A matriz do sistema foi construída pelo esquema de diferenças finitas de 5 pontos, gerando uma matriz bloco-tridiagonal, com:

- -4 na diagonal principal;
- 1 nas posições correspondentes aos vizinhos (cima, baixo, esquerda, direita).

O sistema apresenta 460 elementos não nulos em um total de 10.000 entradas, resultando em 95,4% de zeros, caracterizando uma matriz altamente esparsa.

Em termos de memória, observou-se:

Tabela 14: Comparação de memória entre formatos denso e esparsa

Formato	Memória (MB)
Denso	0,0763
Esparsa (CSR)	0,0056

A representação esparsa consome aproximadamente **13,5 vezes menos memória**, evidenciando sua eficiência para esse tipo de problema.

O sistema foi resolvido por dois métodos:

- Eliminação de Gauss (matriz densa);
- `scipy.sparse.linalg.spsolve` (matriz esparsa).

Os resultados foram:

Tabela 15: Comparação de desempenho entre métodos denso e esparsa

Método	Tempo (ms)	Resíduo
Gauss denso	36,5628	$2,57 \times 10^{-16}$
<code>spsolve</code>	3,7849	$2,88 \times 10^{-16}$

O método esparsa apresentou uma aceleração de aproximadamente **9,7 vezes**, mantendo a mesma ordem de precisão numérica (resíduos da ordem de 10^{-16}).

A diferença entre as soluções foi:

$$\|x_{\text{gauss}} - x_{\text{sparse}}\|_2 = 2,87 \times 10^{-16},$$

indicando que ambas são numericamente equivalentes.

Esses resultados mostram que, para sistemas oriundos de discretizações de EDPs (como o problema de Poisson), o uso de estruturas esparsas é fundamental, pois reduz significativamente o consumo de memória e o tempo de execução, sem comprometer a precisão da solução.

Q8.3 — Implementação vetorizada

Enunciado: Reescreva `fatoracao_lu` usando apenas operações vetorizadas do NumPy (sem laços Python explícitos). Meça o ganho de desempenho para $n = 100$.

Foi implementada a fatoração LU de duas formas:

- (i) utilizando laços explícitos em Python;
- (ii) utilizando operações vetorizadas do NumPy, evitando laços internos.

O teste foi realizado para uma matriz densa aleatória de dimensão $n = 100$, medindo o tempo médio de execução e o erro da fatoração, dado por $\|LU - A\|_F$.

Os resultados obtidos foram:

Tabela 16: Comparação entre implementações com laços e vetorizada

Implementação	Tempo médio (ms)	$\ LU - A\ _F$
LU com laços	11,7765	$4,01 \times 10^{-13}$
LU vetorizada	0,9656	$4,05 \times 10^{-12}$

Observa-se que a versão vetorizada é aproximadamente **12,2 vezes mais rápida** que a implementação com laços. Esse ganho significativo ocorre porque as operações vetorizadas utilizam rotinas otimizadas (BLAS) executadas em C/Fortran, evitando o *overhead* dos loops em Python.

Em relação à precisão, ambas as implementações apresentaram erros muito pequenos, próximos da precisão numérica esperada, indicando que a fatoração foi realizada corretamente em ambos os casos. A pequena diferença entre os erros (10^{-13} vs 10^{-12}) é decorrente de efeitos de arredondamento acumulados durante as operações.

Portanto, a vetorização permite obter grande ganho de desempenho sem comprometer significativamente a precisão, sendo a abordagem mais eficiente para implementação de algoritmos numéricos em ambientes como o NumPy.